Boxing is the cost

An `Integer` takes 16 B to hold a 4 B `int`, plus a 4 B reference to it.

1,000,000 elements, compact headers on	Retained	Per element
JDK `ArrayList<Integer>`, presized	20.0 MB	20 B
EC `IntArrayList` (primitive)	4.0 MB	4 B
JDK `HashSet<Integer>`	48.4 MB	48 B
EC `IntHashSet` (primitive)	8.4 MB	8 B
JDK `HashMap<Integer, Integer>`	48.4 MB	48 B
EC `IntIntHashMap` (primitive)	16.8 MB	17 B

Every `HashMap` entry is one more object

`HashMap` and `HashSet` wrap each entry in a `HashMap$Node`.

- A node is 24 B with compact headers, 32 B without.
- `HashSet` is a `HashMap` inside: 32.4 MB before counting the `Integers`.
- EC `UnifiedSet` keeps the elements in its table: 8.4 MB, 4x less.
- Node plus box: about 48 B of heap per element to hold one `int`.

A stream is an object graph before it runs

An unevaluated pipeline already holds objects, and every operation adds a stage.

Pipeline on an empty list, compact headers on	JDK `Stream`	EC `LazyIterable`
start: `stream()` / `asLazy()`	80 B	16 B
+ `filter` / `select`	144 B	24 B
+ `map` / `collect`	144 B	24 B
+ `distinct`	128 B	16 B
3 steps: filter, map, mapToLong	272 B	96 B